



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 8

“Cronógrafo de 60 segundos”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 20 de febrero de 2025

INTRODUCCIÓN

Cronómetro de 60 segundos

Un cronómetro de 60 segundos es un sistema de temporización que, a diferencia de un contador simple, requiere una base de tiempo extremadamente precisa y una lógica de reinicio (reset) específica al llegar al número 60.

Para entenderlo a fondo, se deben de analizar sus tres pilares fundamentales:

1. La Base de Tiempo

Para que un cronómetro sea útil, cada incremento debe ocurrir exactamente cada 1000 milisegundos.

- **Oscilador de Cristal:** En sistemas con microcontroladores como el ATMEGA8535, se usa un cristal de cuarzo (por ejemplo, 16 MHz).
- **El divisor de frecuencia (Prescaler):** Como 16 millones de pulsos por segundo es demasiado rápido, el hardware utiliza un prescaler para reducir esa frecuencia a algo manejable, y luego un contador interno dispara una interrupción cada vez que se cumple un segundo exacto.

2. La Lógica de Conteo Sexagesimal

A diferencia de un contador de 00 a 99, un cronómetro de 60 segundos debe volver a cero inmediatamente después del segundo 59.

- **Contador de Unidades:** Es un contador MOD-10 estándar (cuenta de 0 a 9).
- **Contador de Decenas:** En lugar de ser un contador MOD-10 (0 a 9), se configura como un contador MOD-6 (0 a 5).
- **El Reinicio:** Mediante compuertas lógicas o una condición **if** en CodeVisionAVR, el sistema detecta cuando las decenas intentan pasar de 5 a 6. En ese instante, se envía una señal de *Clear* a ambos contadores para que el siguiente estado sea 00.

3. Estados de Control (FSM)

Un cronómetro no solo cuenta; debe ser capaz de reaccionar a comandos externos. Esto se maneja mediante una Máquina de Estados Finitos (FSM):

1. **IDLE:** El cronómetro está en 00 y espera la señal de inicio.
2. **RUN:** El contador incrementa cada segundo.
3. **STOP / PAUSE:** Se detiene el paso del tiempo, pero se mantiene el valor actual en el display.
4. **RESET:** Se fuerzan todos los registros a cero, sin importar el estado anterior.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 8, el circuito a armar es el siguiente:

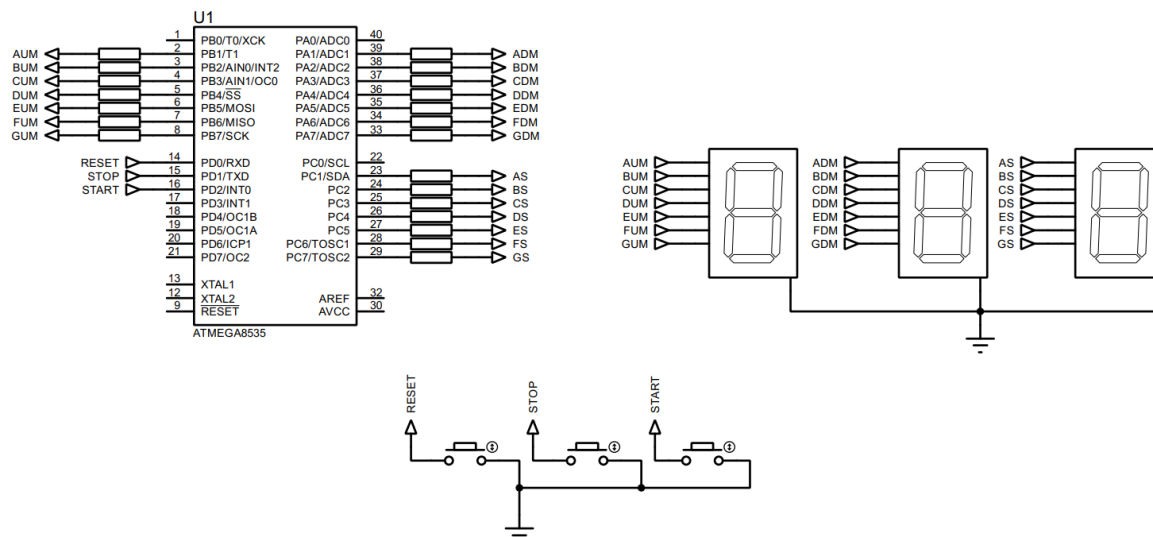


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen el circuito, el registro A, B y C se configuran como de salida (PA1-PA7, PB1-PB7 y PC1-PC7), mientras que el registro D como de entrada (PD0-PD2), activando las resistencias de pull-up.

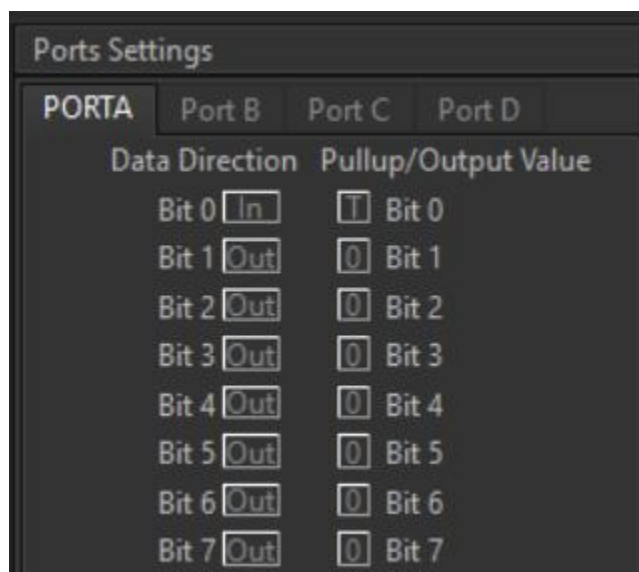
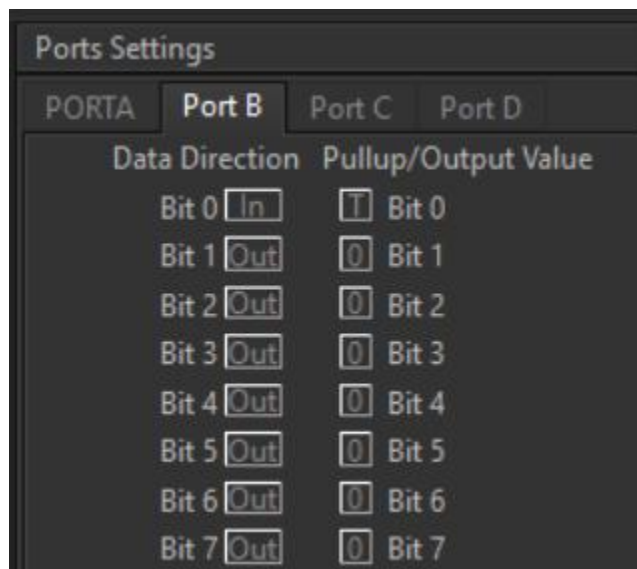
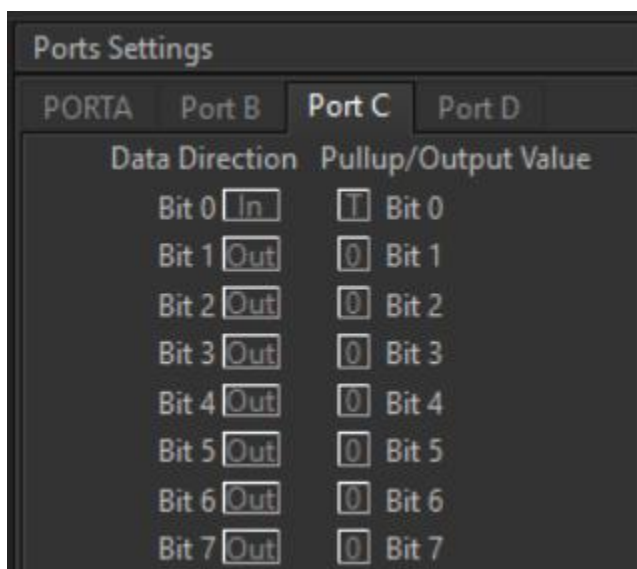


Imagen 2. Configuración del puerto A.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 3. Configuración del puerto B.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 4. Configuración del puerto C.

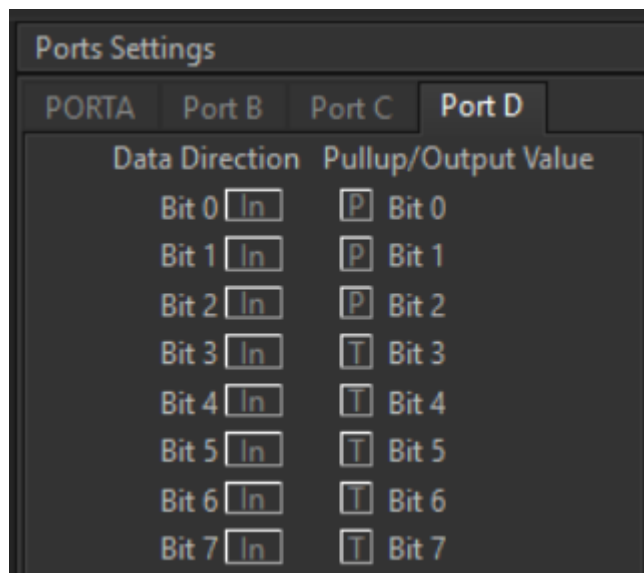


Imagen 5. Configuración del puerto D.

Para poder hacer el cronometro es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc
5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representación hexadecimal para los números del 0 al 9 y del A a la F en hexadecimal.

Código del circuito de CodeVision:

```
1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : PrÃ¡ctica 8 'Cron3metro de 59.9 segundos'
8  Version : 1.0
9  Date    : 17/02/2026
10 Author  : GarcÃ-a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model        : Small
19 External RAM size    : 0
20 Data Stack size     : 128
21 *****/
```

```
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26 #include <stdbool.h>
27 #define RESET_BUTTON PIND.0
28 #define STOP_BUTTON PIND.1
29 #define START_BUTTON PIND.2
```

```

31 // Declare your global variables here
32
33 void main(void) {
34     // Declare your Local variables here
35     unsigned char decena = 0, unidad = 0, decima = 0;
36     bool debe_avanzar = false;
37     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0xe, 0xfe, 0xde};
38
39     // Input/Output Ports initialization
40     // Port A initialization
41     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
42     DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (0<<DDA0);
43     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
44     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
45
46     // Port B initialization
47     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
48     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (0<<ddb0);
49     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
50     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
51
52     // Port C initialization
53     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
54     DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) | (1<<DDC1) | (0<<DDC0);
55     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
56     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
57
58     // Port D initialization
59     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
60     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
61     // State: Bit7=1 Bit6=1 Bit5=1 Bit4=1 Bit3=1 Bit2=1 Bit1=1 Bit0=1
62     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);
63
64     // Timer/Counter 0 initialization
65     // Clock source: System Clock
66     // Clock value: Timer 0 Stopped
67     // Mode: Normal top=0xFF
68     // OC0 output: Disconnected
69     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
70     TCNT0=0x00;
71     OCR0=0x00;
72
73     // Timer/Counter 1 initialization
74     // Clock source: System Clock
75     // Clock value: Timer1 Stopped
76     // Mode: Normal top=0xFFFF
77     // OC1A output: Disconnected
78     // OC1B output: Disconnected
79     // Noise Canceler: Off
80     // Input Capture on Falling Edge
81     // Timer1 Overflow Interrupt: Off
82     // Input Capture Interrupt: Off
83     // Compare A Match Interrupt: Off
84     // Compare B Match Interrupt: Off
85     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
86     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
87     TCNT1H=0x00;
88     TCNT1L=0x00;
89     ICR1H=0x00;
90     ICR1L=0x00;
91     OCR1AH=0x00;
92     OCR1AL=0x00;
93     OCR1BH=0x00;
94     OCR1BL=0x00;

```



```

96 // Timer/Counter 2 initialization
97 // Clock source: System Clock
98 // Clock value: Timer2 Stopped
99 // Mode: Normal top=0xFF
100 // OC2 output: Disconnected
101 ASSR=0<<AS2;
102 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
103 TCNT2=0x00;
104 OCR2=0x00;
105
106 // Timer(s)/Counter(s) Interrupt(s) initialization
107 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
108
109 // External Interrupt(s) initialization
110 // INT0: Off
111 // INT1: Off
112 // INT2: Off
113 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
114 MCUCSR=(0<<ISC2);
115
116 // USART initialization
117 // USART disabled
118 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

120 // Analog Comparator initialization
121 // Analog Comparator: Off
122 // The Analog Comparator's positive input is
123 // connected to the AIN0 pin
124 // The Analog Comparator's negative input is
125 // connected to the AIN1 pin
126 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
127 SFIOR=(0<<ACME);
128
129 // ADC initialization
130 // ADC disabled
131 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
132
133 // SPI initialization
134 // SPI disabled
135 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
136
137 // TWI initialization
138 // TWI disabled
139 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

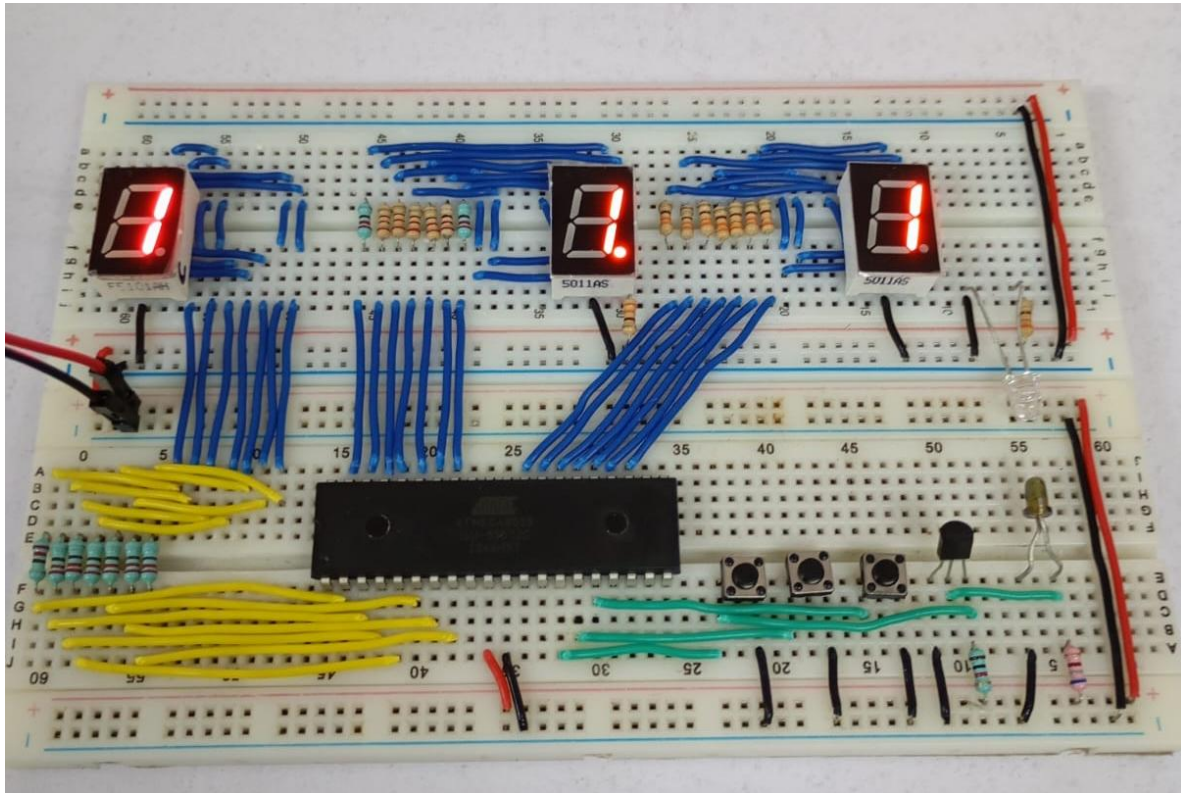
```



```
141 while (1) {
142     // Place your code here
143     if (!START_BUTTON) debe_avanzar = true;
144     if (!STOP_BUTTON) debe_avanzar = false;
145
146     if (!RESET_BUTTON) {
147         decena = 0; unidad = 0; decima = 0;
148         debe_avanzar = false;
149     }
150
151     // Reiniciar las decenas
152     if (decena == 6) debe_avanzar = false;
153
154     if (debe_avanzar) {
155         decima++;
156
157         // Reiniciar los decimales y aumentar unidades
158         if (decima == 10) {
159             decima = 0;
160             unidad++;
161         }
162
163         // Reiniciar las unidades y aumentar decenas
164         if (unidad == 10) {
165             unidad = 0;
166             decena++;
167         }
168     }
169
170     // Obtener los n  meros hexadecimales
171     PORTB = bcd[decena];
172     PORTA = bcd[unidad];
173     PORTC = bcd[decima];
174     delay_ms(100);
175 }
176 }
```

Circuito f  sico

[Aqu   se ponen las fotos del circuito f  sico funcionando]



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

De esta práctica lo más destacable más allá de usar tres displays de forma independiente fue el manejo de las operaciones del contador: RESET, STOP y START. Para ello nos apoyamos de una variable booleana (**debe_avanzar**) que indica si el cronometro debe avanzar o detenerse.

Cuando el botón 'START' se presiona, **debe_avanzar** pasa de **FALSE** a **TRUE** permitiendo que el conteo para las decenas, unidades y decimas se realice de forma normal hasta que **decena** llega a 6, es decir, en cronometro llega a 60 segundos y entonces ahora se pasa de **TRUE** a **FALSE**, esperando un reinicio del cronometro para volver a iniciar el conteo.

Cuando el botón 'RESET' se presiona, tanto las decenas, unidades y decimas vuelven a cero, pero con el cronometro sin avanzar, esperando a que se presione nuevamente el botón 'START', puesto que **debe_avanzar** se mantiene en **FALSE**.

Cuando el botón 'STOP' se presiona, **debe_avanzar** se mantiene en **FALSE** evitando que el conteo avance, permitiendo observar en los displays el valor del conteo al momento de detener el cronometro, esperando a ser reanudado con 'START' desde el último valor del conteo.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un cronometro de 60 segundos usando 3 displays independientes, una para contabilizar las decenas, otro para las unidades y el ultimo para las décimas. Cada display tiene su propio funcionamiento pues actúa de acuerdo con el rol asignado.

En el caso de las decenas, este solo llega hasta 6 pues el contador solo permite cronometrar hasta 60 segundos, una vez llegado a ese punto el conteo se detiene. Para el caso de las unidades y décimas, estos simplemente deben llegar hasta 10 e inmediatamente deben reiniciarse para reiniciar el ciclo.

Por ultimo y no menos importante, para toda esta lógica de programación se utilizó una variable booleana que denominamos como **“debe_avanzar”**, esta se encarga de indicar cuando es que el conteo debe continuar o detenerse, y para ello también implementamos 3 operaciones indispensables para un cronometro: RESET, START y STOP. Cada una de estas cumple un rol diferente en el circuito y se apoyan de la variable **“debe_avanzar”** para su correcto funcionamiento.

BIBLIOGRAFÍA

(s.f.). Mod-60-Counter. Electronic Course. Recuperado el 19 de febrero de 2026.
<https://electronics-course.com/>

(s.f.). Atmel AVR Timers. AVR Tutorials. Recuperado el 19 de febrero de 2026.
<https://www.avr-tutorials.com/timers/programming-avr-8-bit-timer-counter0-standard-mode>